Reviewer Pack — Tropicalized K-theory Rank vs Communication Complexity for k…

Entry 692c25bf5ab6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 04:21:01 UTC

Tropicalized K-theory Rank vs Communication Complexity for k-Clique

Entry ID: `692c25bf5ab6` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 04:21:01 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (K-theory) **Field B** (complexity object): Communication Complexity

Statement:

> ['For every graph G with n vertices, let K(G) be the tropicalization of its K-group and denote by τ(K(G)) the rank of the tropicalized K-group. Then for any communication protocol P for k-Clique on G, the total number of bits exchanged in P is at least τ(K(G)).', 'Equivalently, there exists a constant c > 0 such that for all graphs G with n vertices, if P is a communication protocol for k-Clique on G, then the number of bits exchanged by P is at least cτ(K(G)).']

Rationale (proposer's reasoning):

> ['K-theory has been successfully applied to study topological properties of spaces and algebras. Tropicalizing K-theory may reveal new structural insights into combinatorial objects like graphs, potentially providing lower bounds for communication complexity problems.', 'The rank of the tropicalized K-group captures essential information about the algebraic structure of the graph. This invariant could be used to derive lower bounds on communication complexity by relating it to the structure of the graph.']

Taxonomy category: `TROPICAL_KTHEORY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f1ec0df7bd9c618c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a given graph G with n vertices, if the communication protocol P for k-Clique on G has at least cτ(K(G)) bits exchanged, where τ(K(G)) is the tropicalization rank of K(G) and c is a constant greater than zero.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropicalization K-theory rank" AND "communication complexity" AND k-clique - "K-group tropicalization" related to "protocol communication complexity" - "bit exchange in protocols" FOR k-Clique IN relation TO "K-theory rank tropicalization"

Top relevant hits considered: - [http://arxiv.org/abs/cs/0608100v1] Similarity of Semantic Relations - [http://arxiv.org/abs/0810.1207v1] A Layered Grammar Model: Using Tree-Adjoining Grammars to Build a Common Syntactic Kernel for Related Dialects - [http://arxiv.org/abs/1310.8154v3] Characteristic cohomology of the infinitesimal period relation - [http://arxiv.org/abs/2104.13209v2] Parallel K-Clique Counting on GPUs - [http://arxiv.org/abs/1610.09089v1] The Dynamic Descriptive Complexity of k-Clique - [http://arxiv.org/abs/cond-mat/0610298v1] The critical point of k-clique percolation in the Erdos-Renyi graph

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda x: abs(A[x][i]))
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                continue
            for j in range(n):
                A[i][j] /= A[i][i]
            for k in range(m):
                if k != i and A[k][i] != 0:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A
```

```
def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0 for _ in range(p)] for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if n == 1:
        return A[0][0]
    det = 0
    for i in range(n):
        submatrix = [row[:i] + row[i+1:] for row in A[1:]]
        det += (-1) ** i * A[0][i] * determinant(submatrix)
    return det

def k_group_rank(G):
    n = len(G)
    adjacency_matrix = [[G[i][j] if G[i][j] == 1 else 0 for j in range(n)] for i in range(n)]
    laplacian_matrix = [[sum(adjacency_matrix[i]) - adjacency_matrix[i][j] if i == j else -adj
    return sum(1 for row in gaussian_elimination(laplacian_matrix) if any(row))

def k_clique_protocol(G, k):
    n = len(G)
    visited = [False] * n
    def dfs(node, path):
        if len(path) == k:
            return True
        visited[node] = True
        for neighbor in range(n):
            if G[node][neighbor] and not visited[neighbor]:
                if dfs(neighbor, path + [neighbor]):
                    return True
        visited[node] = False
        return False
    return any(dfs(i, [i]) for i in range(n))

n_values = [10, 15, 20, 25, 30]
results = []

for n in n_values:
    G = [[random.randint(0, 1) if i != j else 0 for j in range(n)] for i in range(n)]
    K_G_rank = k_group_rank(G)
    protocol_bits = len(k_clique_protocol(G, 3)) * 2  # Assuming each bit exchange is counted
    results.append({
        "metric_name": "protocol_bits",
        "metric_value": protocol_bits,
        "instances_tested": 1,
        "conjecture_holds": protocol_bits >= K_G_rank,
        "counterexample": "" if protocol_bits >= K_G_rank else f"Graph with n={n} and rank {K_
    })
```

```
    mean_metric = sum(result["metric_value"] for result in results) / len(results)
    std_metric = math.sqrt(sum((result["metric_value"] - mean_metric) ** 2 for result in results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    return {
        "seed": seed,
        "mean_metric": mean_metric,
        "std_metric": std_metric,
        "support_fraction": support_fraction
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric = sum(result["mean_metric"] for result in results) / len(results)
    std_metric = math.sqrt(sum((result["mean_metric"] - mean_metric) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["support_fraction"] == 1) / len(resul

    if all(result["support_fraction"] == 1 for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
    elif any(result["support_fraction"] < 0.8 for result in results):
        print("RESULT: FALSIFIED counterexample=\"not enough seeds supporting\" first_failing_seed
    else:
        print(f"RESULT: INCONCLUSIVE reason=budget_exceeded n_tested={len(results)}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d24512a0.py", line 111, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d24512a0.py", line 84, in run_trial
    K_G_rank = k_group_rank(G)
               ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d24512a0.py", line 62, in k_group_rank
    return sum(1 for row in gaussian_elimination(laplacian_matrix) if any(row))
                           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d24512a0.py", line 25, in gaussian_elim
    A[i], A[max_row] = A[max_row], A[i]
                       ~^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means we cannot verify the conjecture with the provided information. | next: Re-run the test ensuring it completes without crashing and provides results to assess the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12235
2	preregistration	ollama_remote	glm4:latest	0	0	5564
3	novelty	ollama_remote	glm4:latest	0	0	4730
4	novelty	ollama_remote	glm4:latest	0	0	5609
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13507
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10608
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11711
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13648
9	judge	ollama_remote	glm4:latest	0	0	8671

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 86284 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/692c25bf5ab6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/692c25bf5ab6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/692c25bf5ab6.tar.gz` (if generated)